

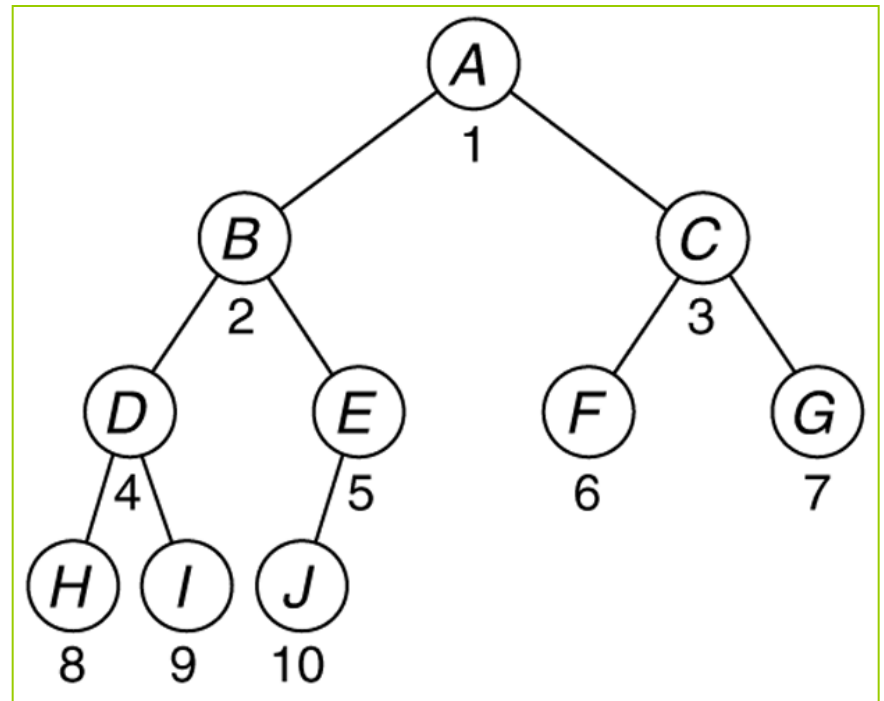
HEAP

Properties of Heaps

- ◆ Heaps are binary trees that are ordered from bottom to top, so that a traversal along any leaf-to-root path will visit the keys
 - in ascending order: max heaps : $\text{key}(\text{parent}) \geq \text{key}(\text{child})$
 - in descending order: min heaps : $\text{key}(\text{parent}) \leq \text{key}(\text{child})$
- ◆ Heaps are (1) to implement priority queues,
(2) to implement the Heap sort algorithm.
- ◆ A heap is a complete binary tree except the bottom level adjusted to the left.

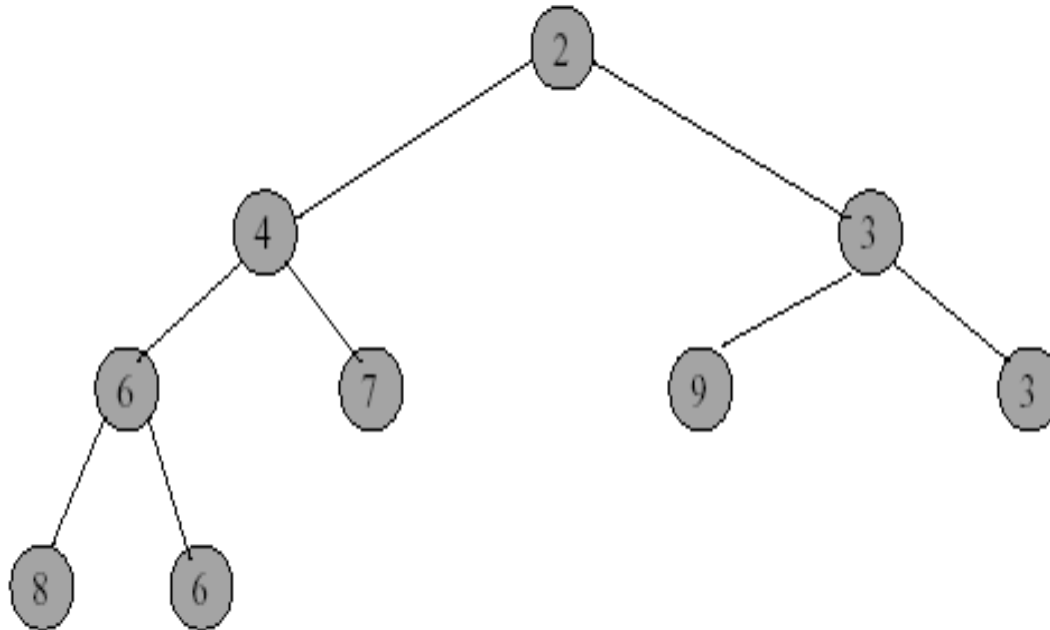
Heap

Heap shape:



Min Heap with 9 Nodes

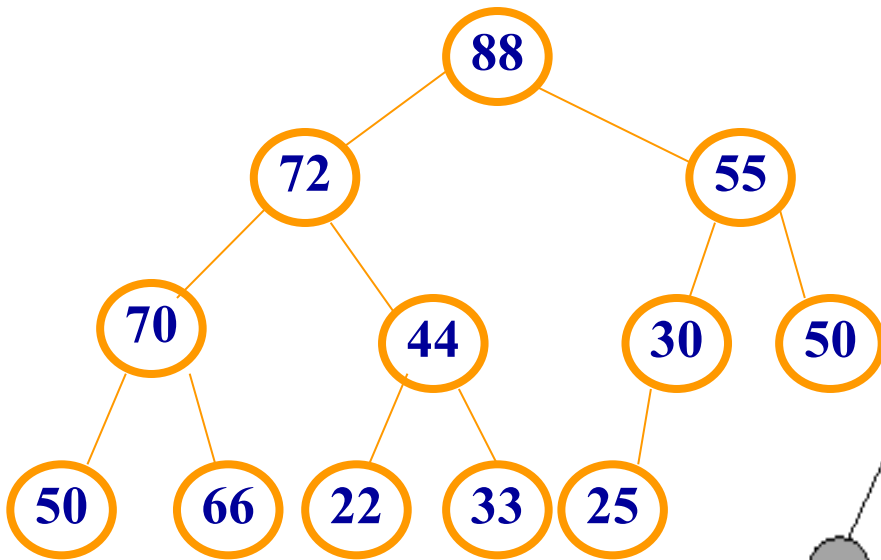
A **minimal heap** (descending heap) is an almost complete binary tree in which the value at each parent node is less than or equal to the values in its child nodes.



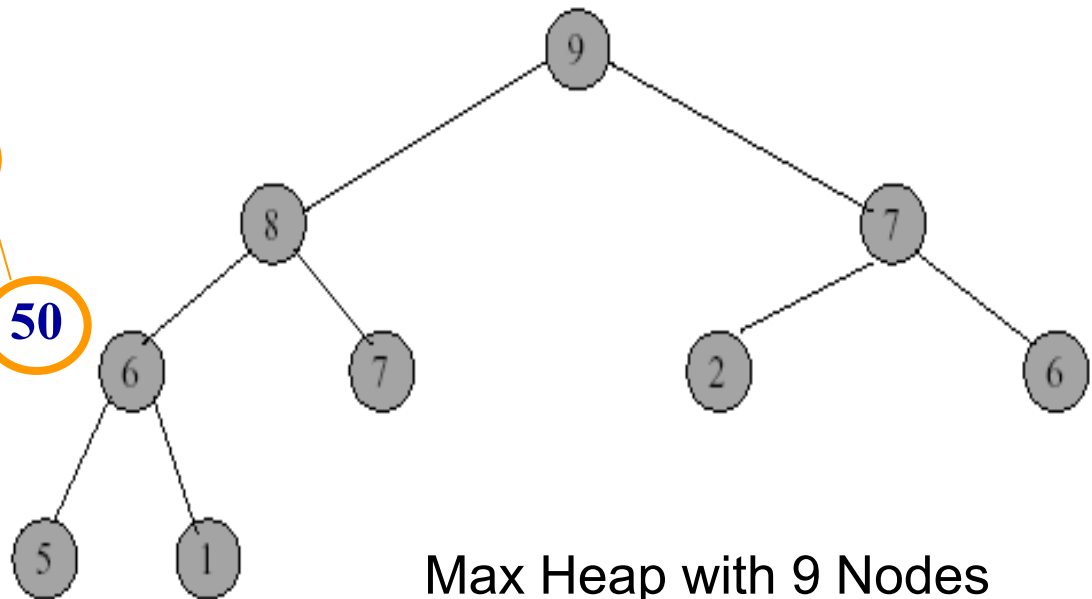
Complete binary tree with 9 nodes

Max Heap

A **Max heap** (Ascending heap) is an almost complete binary tree in which the value at each parent node is greater than or equal to the values in its child nodes.



Max Heap with 12 Nodes



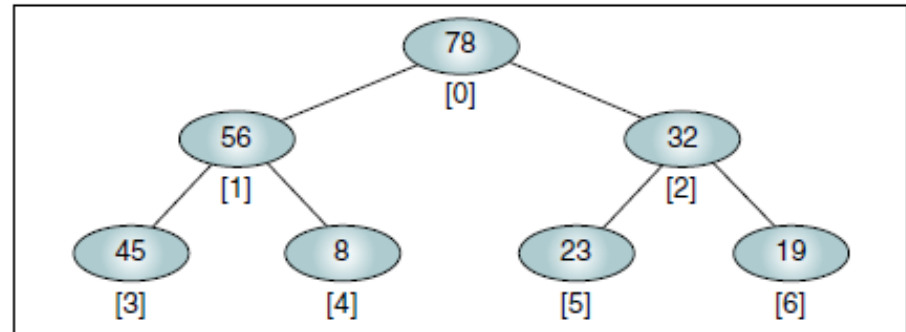
Max Heap with 9 Nodes

Heap Implementation

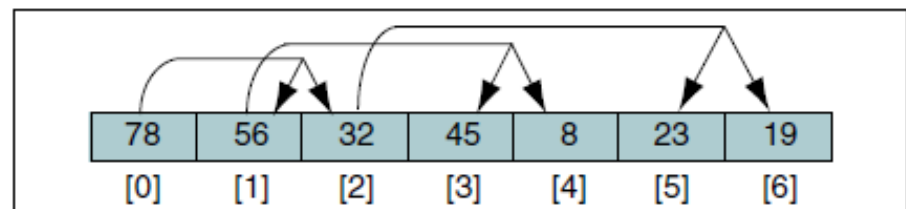
We can use an array (due to the regular structure or completeness of binary tree).

For a node N with location i, the following factors can be calculated.

- Left child $\rightarrow$ at index $2 * i + 1$
- Right child $\rightarrow$ at index $2 * i + 2$
- Parent $\rightarrow$ at index $\text{floor}((i - 1) / 2)$

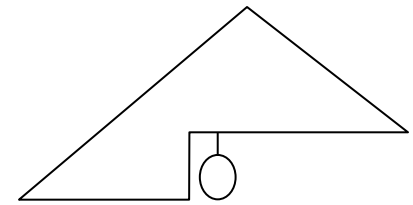


(a) Heap In Its logical form

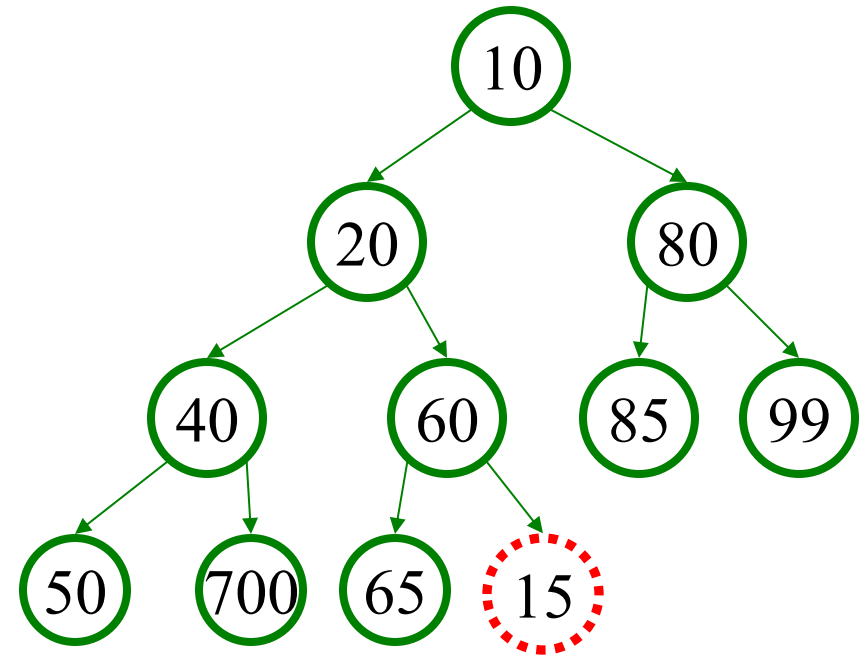
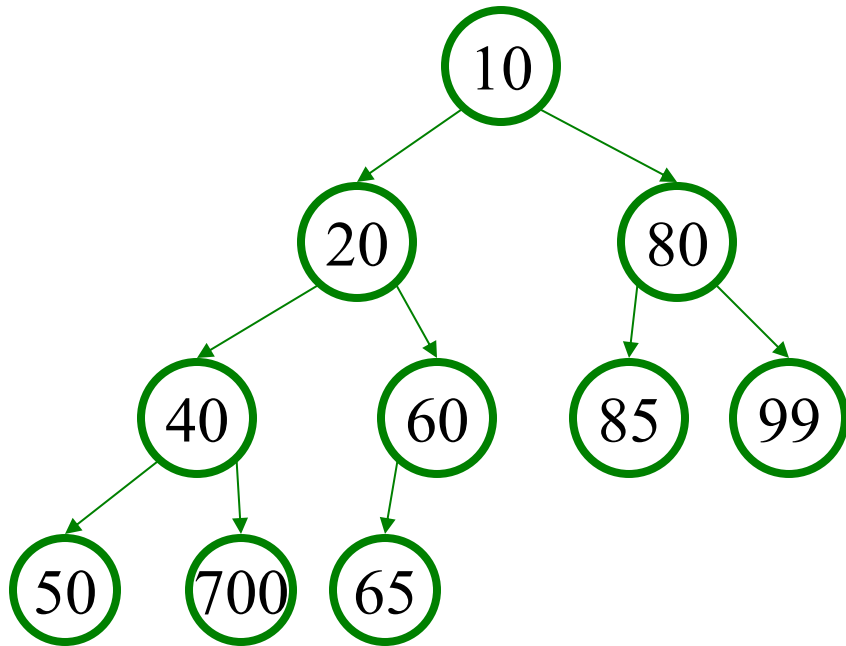


(b) Heap In an array

Adding to a min heap

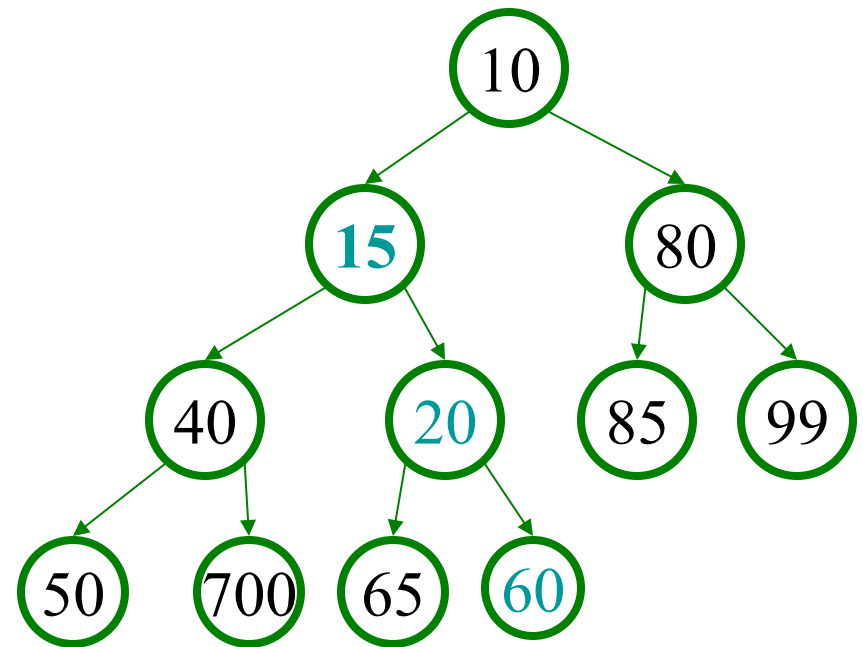
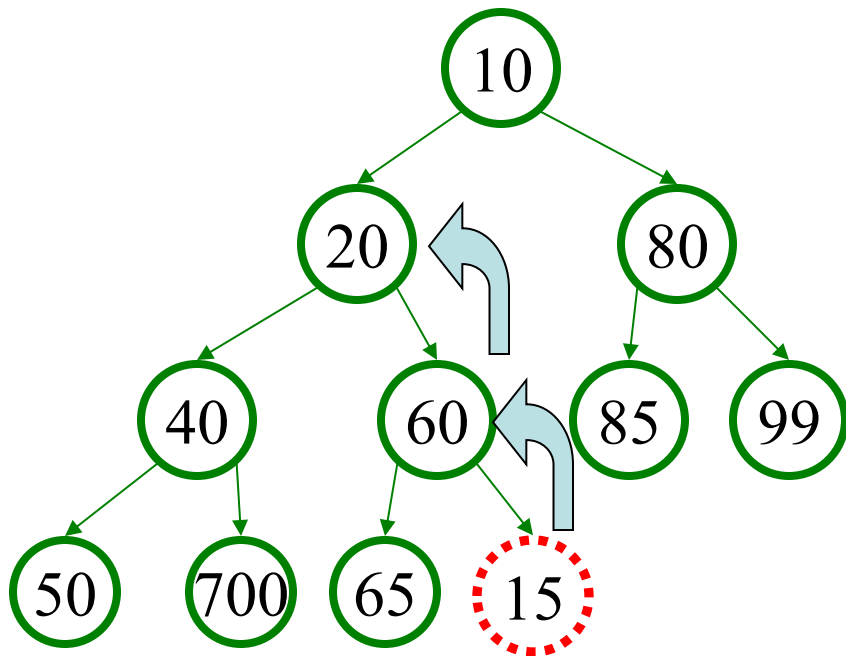


- when an element is added to a heap, it should be initially placed as the rightmost leaf (to maintain the completeness property)
 - heap ordering property becomes broken!



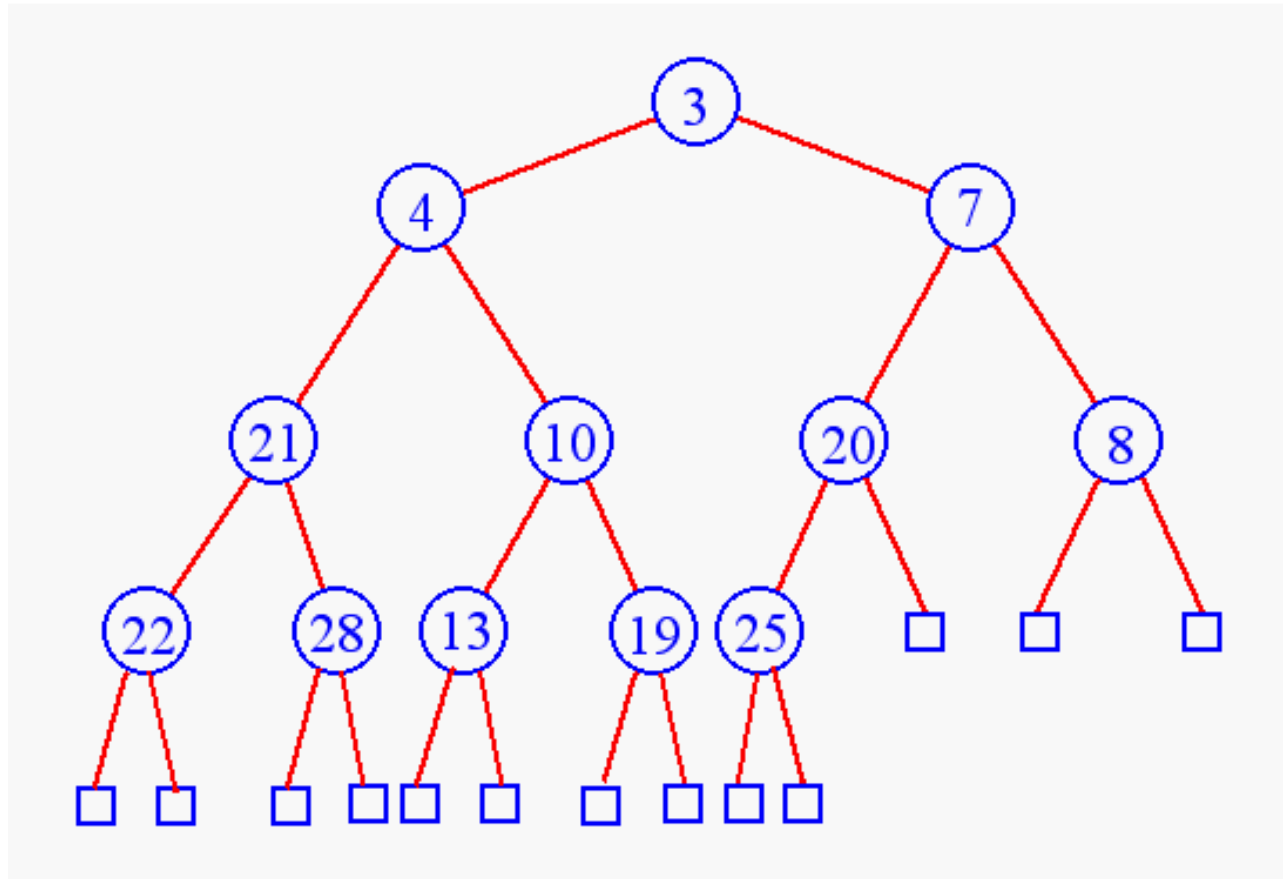
Adding to a min heap, cont..

- To restore heap ordering property, the newly added element must be shifted upward ("bubbled up") until it reaches its proper place
 - bubble up by swapping with parent



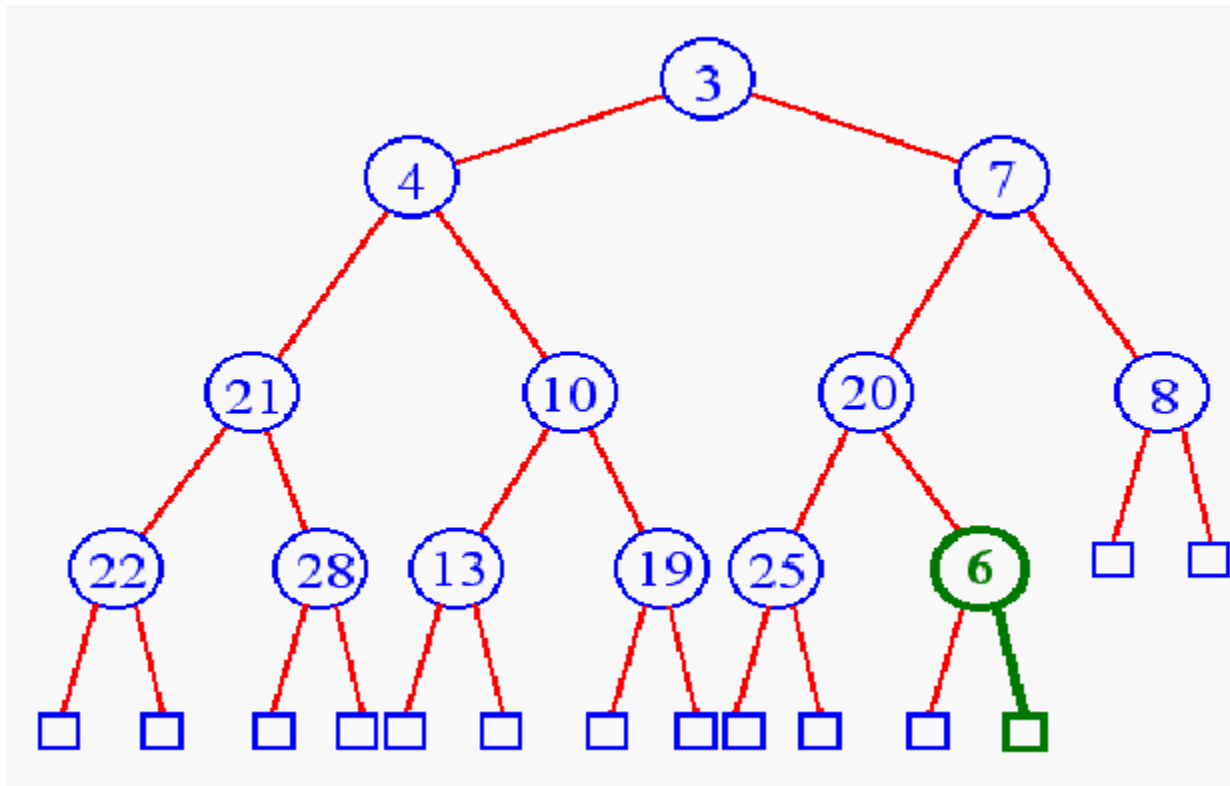
Adding to a min heap, cont..

- Insert 6



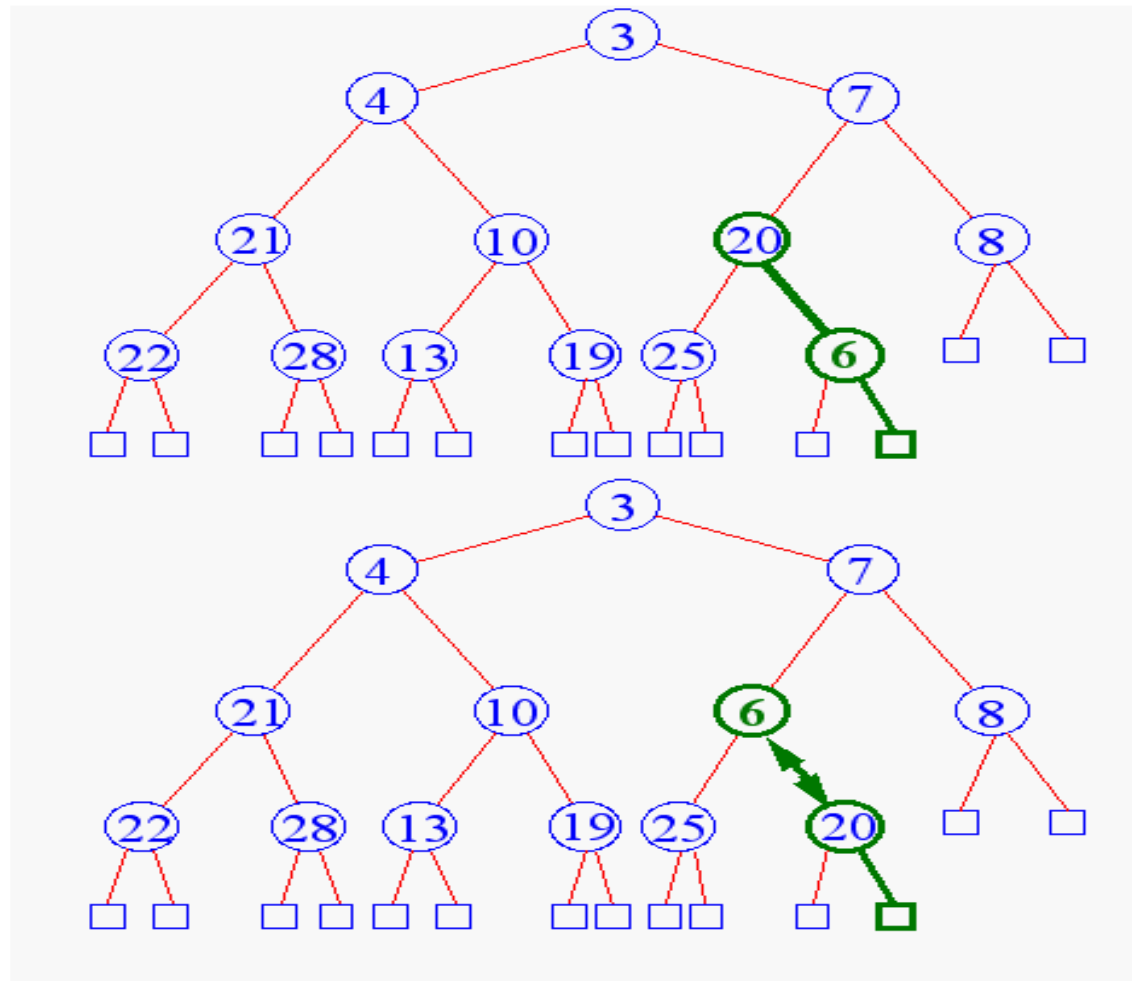
Adding to a min heap, cont..

- Add key in next available position

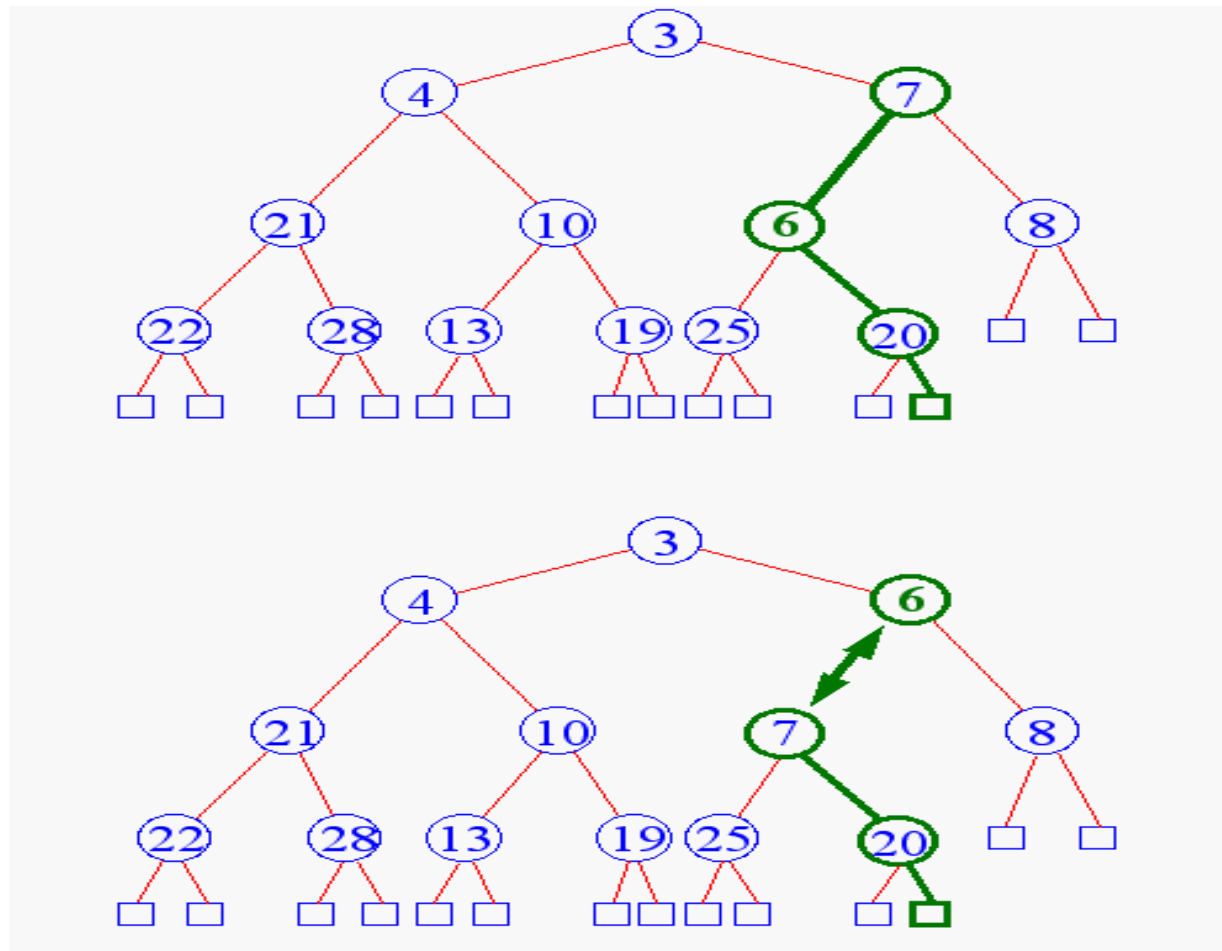


Adding to a min heap, cont..

- Begin Unheap

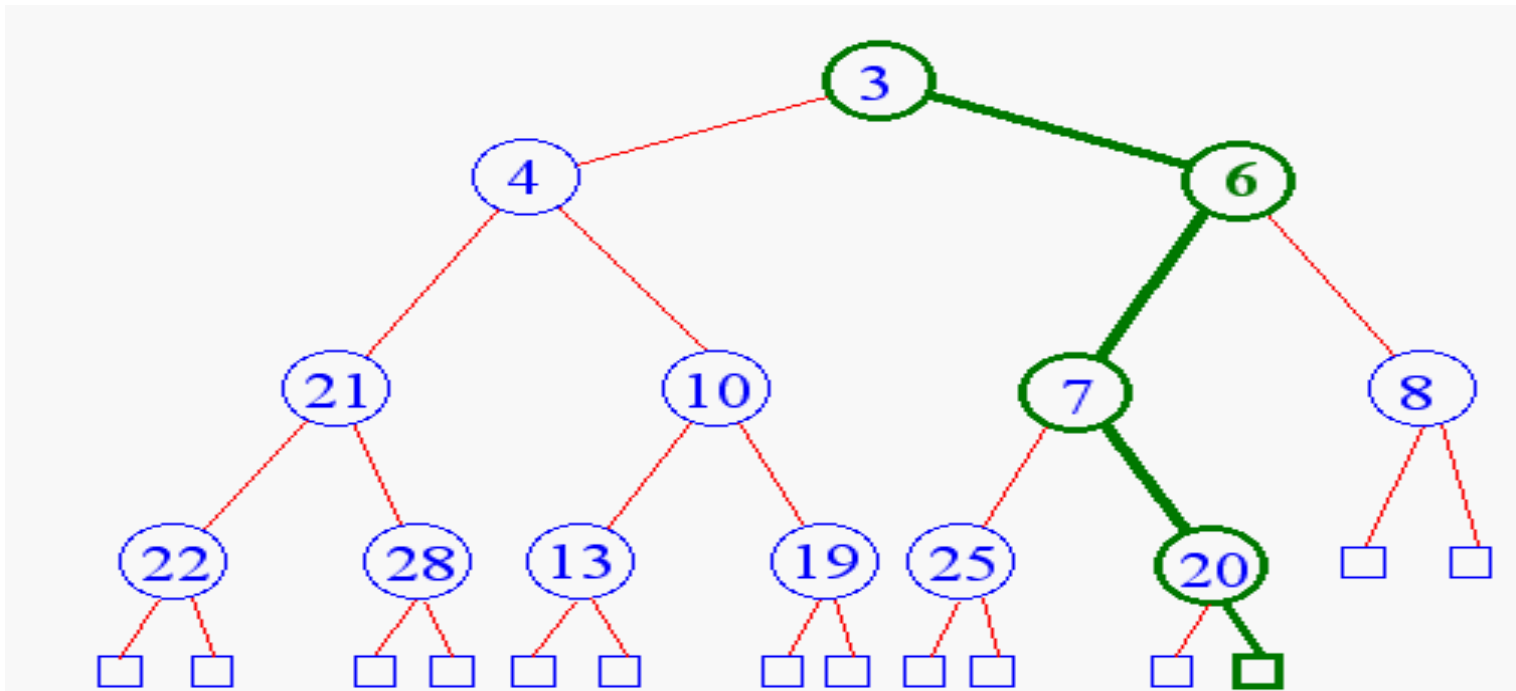


Adding to a min heap, cont..



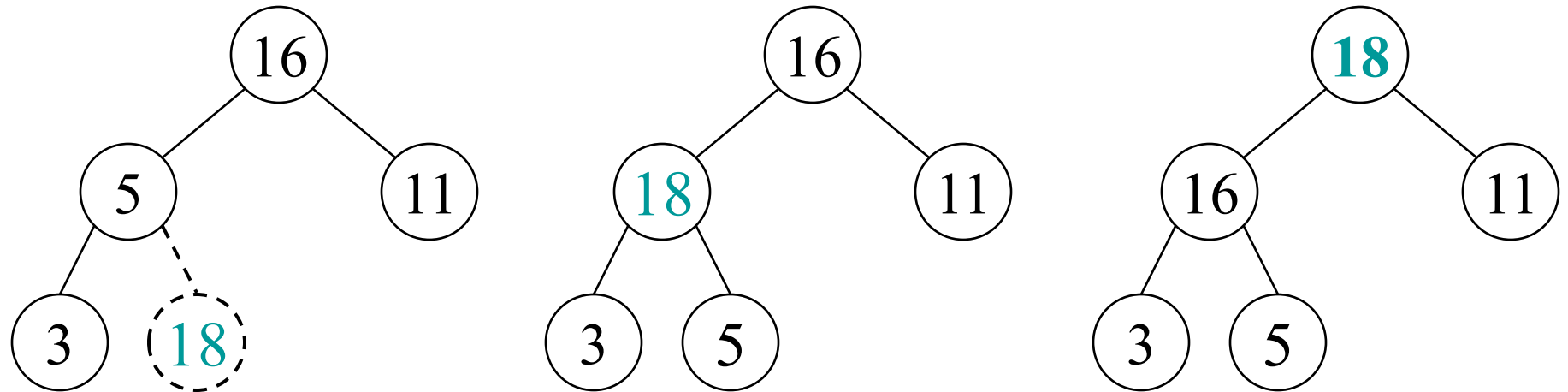
Adding to a min heap, cont..

- Terminate unheap when
 - reach root
 - key child is greater than key parent



Adding to a max-heap

- same operations, but must bubble up *larger* values to top

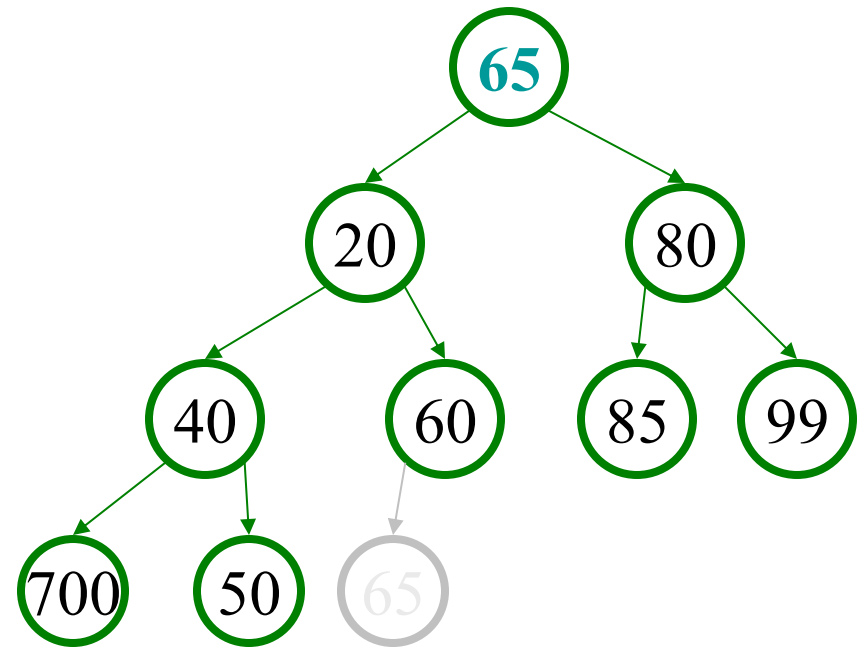
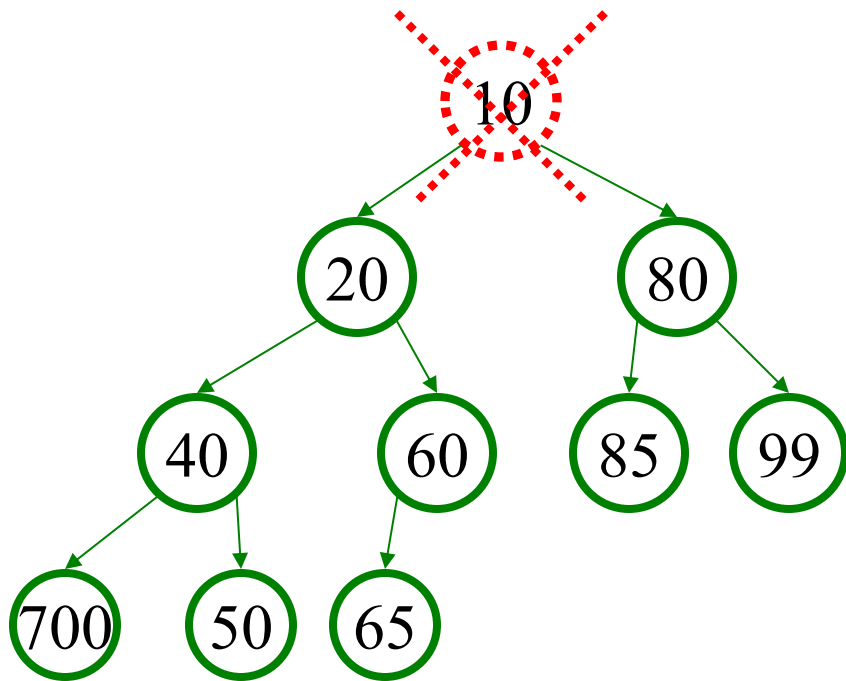


Removing from a Heap

- We always remove the item from the root. That way we always get the smallest item. The problem is then how to adjust the binary tree so that we again have a heap (with one less item).
- The algorithm works like this:
 - Remove the root item and replace it with the last item (target).
 - Check the path from the root to a leaf for the correct position for this target.
 - The path is chosen by always selecting the smaller child at each node.

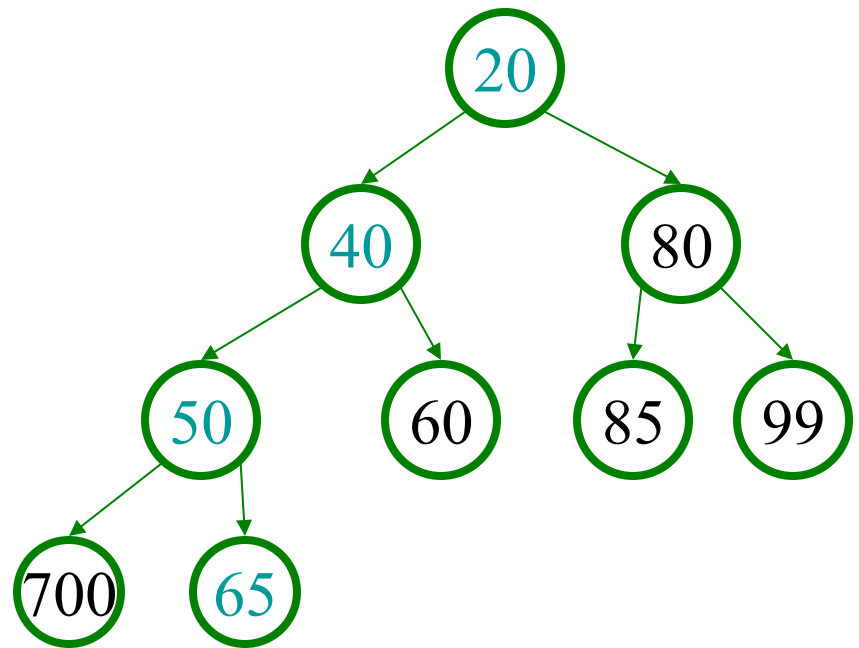
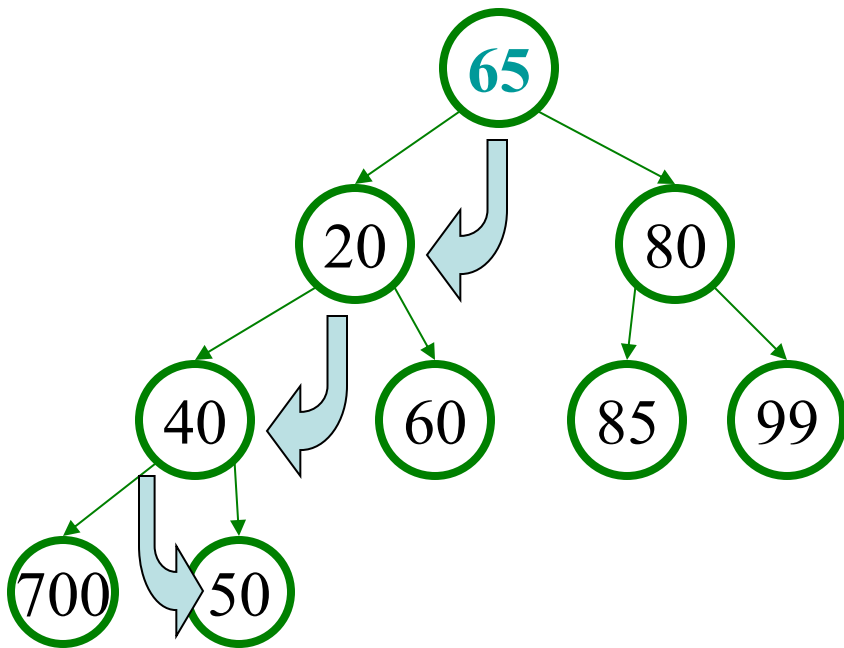
Removing from a min-heap

- min-heaps only support `remove` of the min element (the root)
 - must remove the root while maintaining heap completeness and ordering properties
 - intuitively, the last leaf must disappear to keep it a heap
 - initially, just swap root with last leaf (we'll fix it)



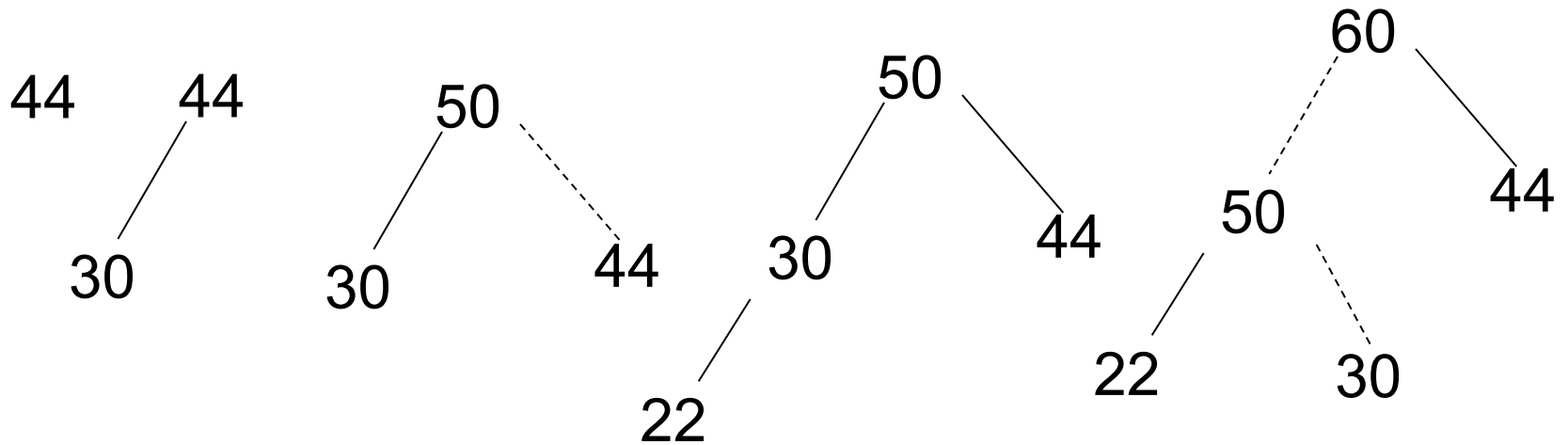
Removing from heap, cont..

- must fix heap-ordering property; root is out of order
 - shift the root downward ("bubble down") until it's in place
 - swap it with its smaller child each time

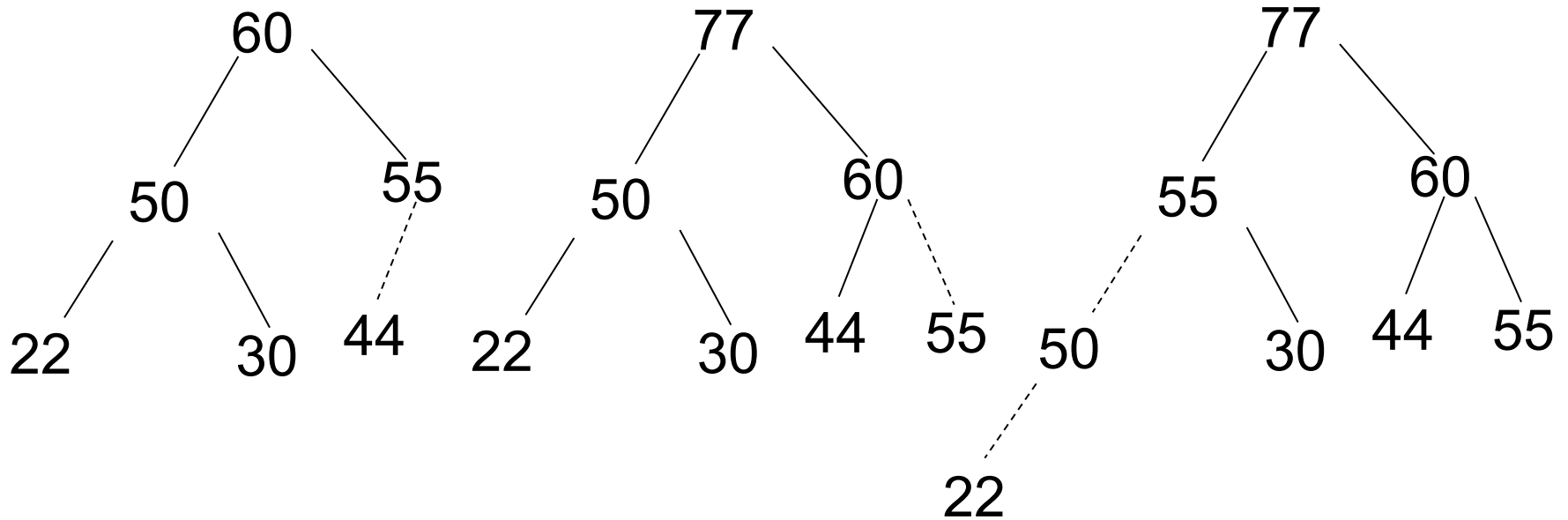


Building a Heap

- Build a heap H from the following list of numbers:
44, 30, 50, 22, 60, 55, 77, 55

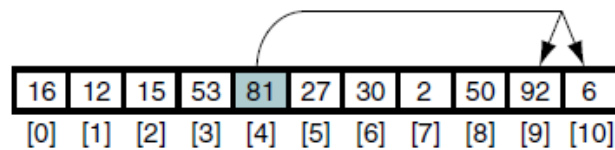


Building a Heap

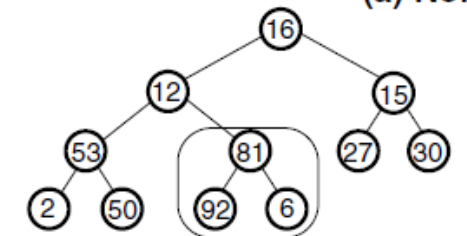


Building a Heap

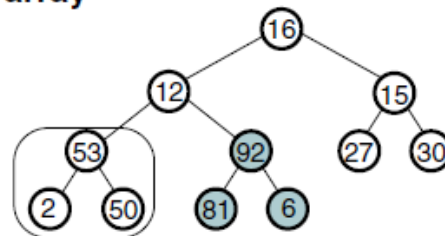
- Insert one by one: Insert each element into an initially empty heap and continuously adjust.
 - Complexity: Doing this n times $\rightarrow O(n \log n)$ total.
- From a list of elements:
 - You start heapifying from the bottom non-leaf nodes.
 - Starting index: $(n / 2) - 1$
 - Then, working up the tree to the root
 - Complexity: $O(n)$, not $O(n \log n)$.



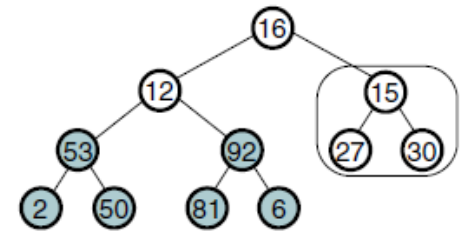
(a) Nonheap array



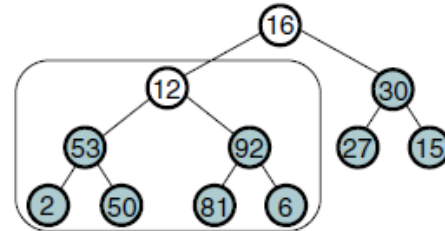
(b) Before first reheap down



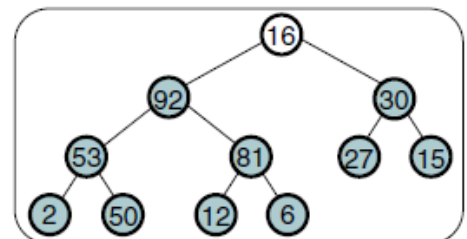
(c) After first reheap down



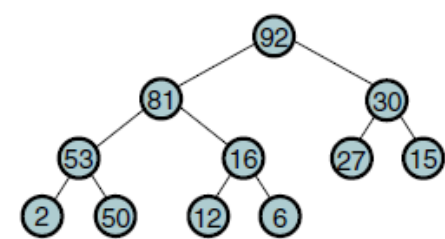
(d) After second reheap down



(e) After third reheap down



(f) After fourth reheap down



(g) Complete heap

// Function to heapify a subtree rooted at index i

```
void heapify(int arr[], int n, int i) {
    int largest = i;    // Initialize largest as root
    int left = 2 * i + 1; // left child index
    int right = 2 * i + 2; // right child index

    // If left child exists and is greater than root
    if (left < n && arr[left] > arr[largest])
        largest = left;

    // If right child exists and is greater than largest so far
    if (right < n && arr[right] > arr[largest])
        largest = right;

    // If largest is not root
    if (largest != i) {
        // Swap root and largest
        int temp = arr[i];
        arr[i] = arr[largest];
        arr[largest] = temp;

        // Recursively heapify the affected subtree
        heapify(arr, n, largest);
    }
}
```

// Function to build a max heap

```
void buildHeap(int arr[], int n) {
    // Index of the last non-leaf node
    int startIdx = (n / 2) - 1;

    for (int i = startIdx; i >= 0; i--) {
        heapify(arr, n, i);
    }
}
```

Applications of Heaps

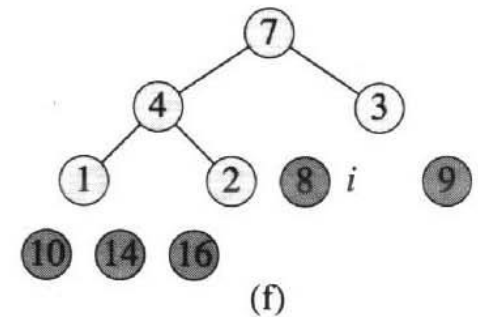
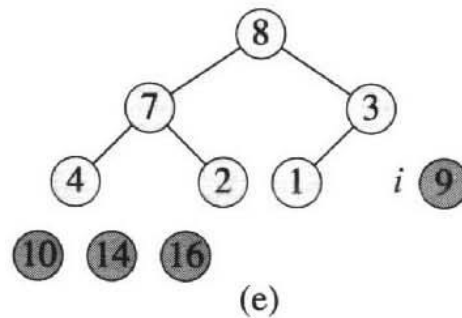
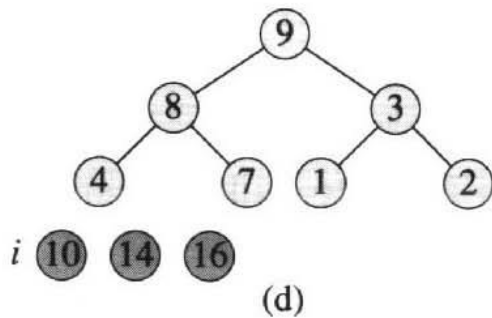
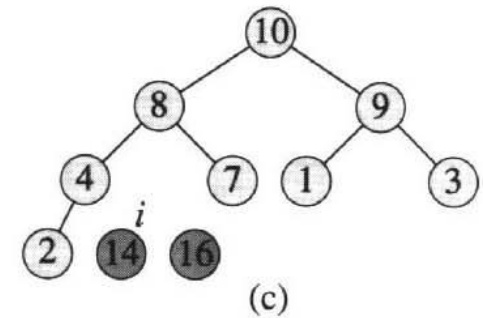
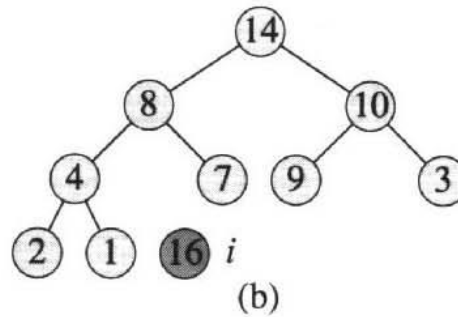
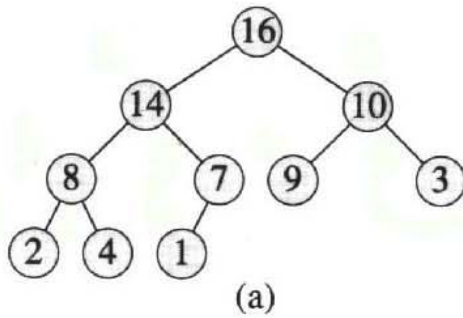
- Sorting (heap sort)
- A heap is the standard, most efficient way to implement a priority queue. If you need a Max Priority Queue, you use a Max-Heap. If you need a Min Priority Queue, you use a Min-Heap.
- Machine scheduling
- Huffman codes

Heapsort

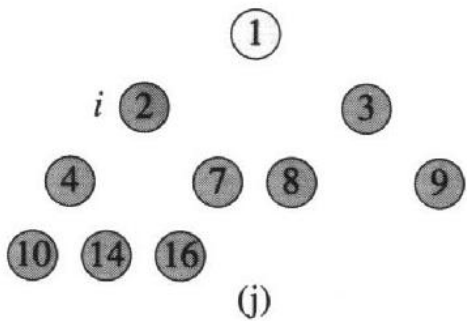
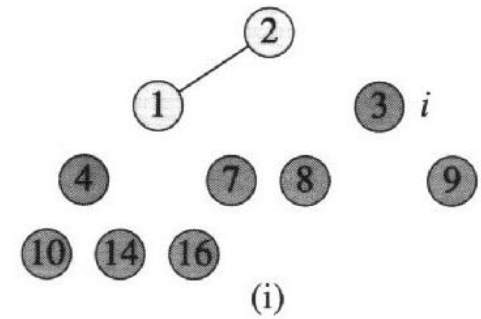
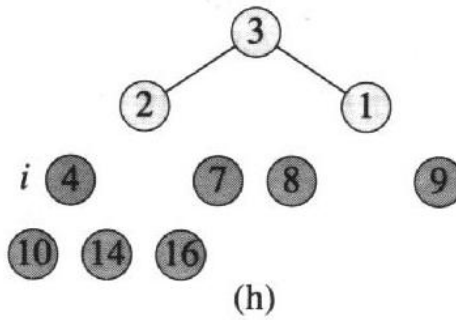
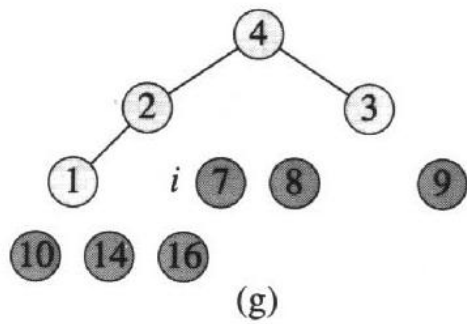
Suppose an array with N elements is given.

- Step 1: Build a heap H out of the elements of A
- Step 2: Repeatedly delete the root element of H
- For sorting in descending order, a max heap must be created. For ascending order, a min heap must be created.
- Heap Sort takes $O(n \log n)$ times.

Heap Sort



Heap Sort



A

1	2	3	4	7	8	9	10	14	16
---	---	---	---	---	---	---	----	----	----

(k)

Exercises

Chapter 9 (HEAP)

Exercises: 2, 3, 5, 6, 7, 12, 16, 17, 18

Source: Data Structures: A Pseudocode Approach Using C, 2nd ed